4.3 Playing with ROS2

Time to play with realistic sensors! 🤖 In notebook 4.2 we built our own, very simple, laser scanner simulator. Now we are going to work with the sensors of a robot simulated by MVSim, through **ROS2**: what they publish, what their messages contain, where they are mounted on the robot and how noisy they are, connecting all of it with the sensor models of notebooks 4.1 and 4.2.

By the end of this notebook you will be able to configure the sensors of a simulated robot, tell what each one publishes, read the content of a laser scan, measure its noise, and find out where each sensor is mounted.

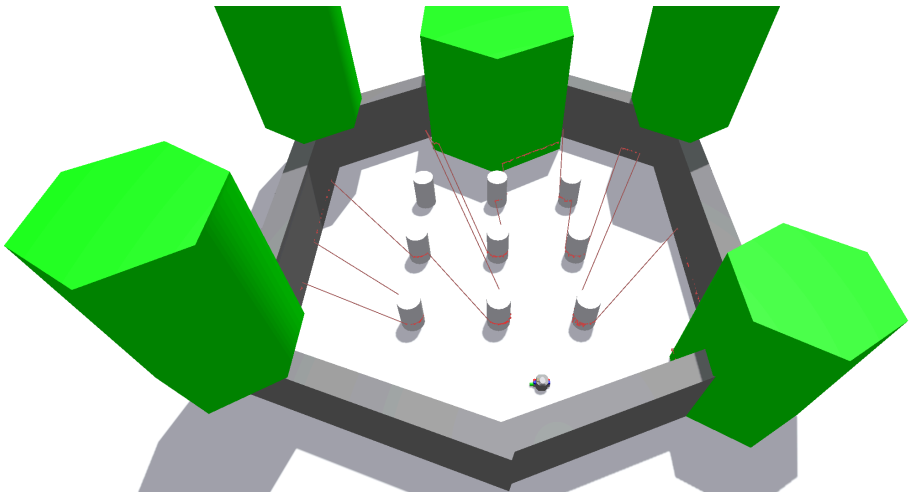


Fig. 1. The TurtleBot3 Burger robot in the TurtleBot3 World of MVSim, together with the beams of its 2D laser scanner (in red).

4.3.1 How to work with this notebook

This notebook is not meant to be run. ROS2 is a set of programs (*nodes*) running at the same time on your own machine, so this notebook plays the role of a **lab guide and report**: keep it open while you work on your terminals, and write your answers back into it:

1. **Commands**: paste the command you used (and, when asked, its output) into the markdown code blocks below.
2. **Screenshots**: a capture of the simulator, of RViz2 or of a terminal.

Assumed setup. The same as in notebook 2.1: Ubuntu 24.04, ROS2 Jazzy and MVSim, installed as a ROS2 package (`ros-jazzy-mvsim`).

Tip: you will modify an MVSim world file several times. Make a backup copy of it before your first change (e.g. `sudo cp demo_turtlebot_world.world.xml`

`demo_turtlebot_world.world.xml.bak`), so you can always go back to a working version.

4.3.2 The TurtleBot3 world

In this notebook we are going to work with the **`demo_turtlebot_world`** world of MVSim. Its robot is a TurtleBot3 Burger, a small differential drive robot by ROBOTIS widely used for teaching and research with ROS, and the world reproduces the *TurtleBot3 World*, the arena with columns where it is usually tested.

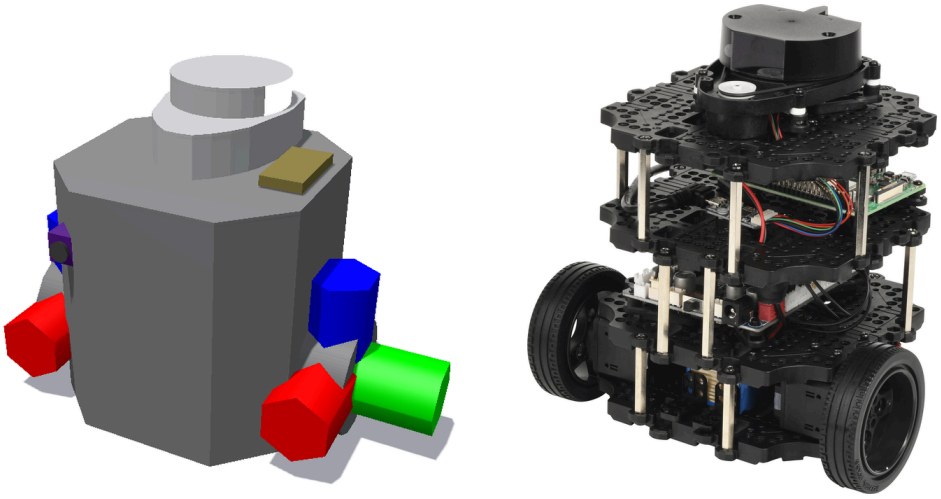


Fig. 2. The TurtleBot3 Burger in MVSim (left) and the real robot (right). In both, the 2D laser scanner is on top.

The robot is defined in the vehicle class `definitions/turtlebot3_burger.vehicle.xml`, which mounts three sensors when its `default_sensors` argument is true: a 360° **2D laser scanner** (simulated as a RPLIDAR A2), an **RGB camera** and an **IMU**. In this notebook we are going to focus on the laser scanner, although we will also keep the IMU. Besides, the camera is a Full HD one (1920 × 1080 pixels) running at 30 images per second, which the simulator renders through the GPU: on a computer without a decent GPU, or inside a virtual machine, it can make the simulation crawl. So, before starting, we are going to remove it.

ASSIGNMENT 1: Preparing the world

As with any vehicle class, `turtlebot3_burger.vehicle.xml` is shared by every world of the system installation, so we are not going to touch it. Instead, we will switch off the default sensors of the class from the world file, and mount our own sensors on the robot there.

What to do?

1. Find where the world file `demo_turtlebot_world.world.xml` was installed in your system, and open it with your favourite editor. Since it belongs to the system installation, you will need `sudo` to save your changes. *Hint: it lives in the `mvsim_tutorial` directory of the installed `mvsim` package, and `ros2 pkg prefix` tells you where a package was installed.* Write below the commands you used.

```
# Dónde está instalado el paquete mvsim
ros2 pkg prefix mvsim
# /opt/ros/jazzy

# El mundo está en share/mvsim/mvsim_tutorial del paquete
cd $(ros2 pkg prefix mvsim)/share/mvsim/mvsim_tutorial
ls | grep turtlebot
# demo_turtlebot_world.launch.py
# demo_turtlebot_world_ros2.rviz
# demo_turtlebot_world.world.xml
# ros1_turtlebot_world.launch

# Copia de seguridad y edición con sudo (el archivo es de la
instalación del sistema)
sudo cp demo_turtlebot_world.world.xml
demo_turtlebot_world.world.xml.bak
sudo nano demo_turtlebot_world.world.xml
```

2. Find the `<include>` of the `turtlebot3_burger` vehicle class and set its `default_sensors` argument to `false`, so the robot arrives with no sensors at all. Paste below the piece of XML you changed.

```
<include file="../definitions/turtlebot3_burger.vehicle.xml"
    default_sensors="false"
/>
```

3. Now mount the sensors on the robot from the world file. Inside the `<vehicle name="r1">` tag, right after its `<init_pose>`, add the following block (no need to type it, just copy it). The laser scanner and the IMU are the same as in the vehicle class, and so are their names, so the RViz2 configuration of the demo keeps working (only its camera panel will stay empty).

```
<!-- 2D laser scanner (RPLIDAR A2), as in the TurtleBot3 class -->
<include file="../definitions/rplidar-a2.sensor.xml"
  raytrace_3d="true"
  sensor_x="-0.032" sensor_z="0.171" sensor_yaw="0"
  sensor_hz="10.0"
  sensor_name="laser1"
/>

<!-- IMU, as in the TurtleBot3 class -->
```

```
<include file="../definitions/imu.sensor.xml"
  sensor_x="0.0" sensor_y="0.05" sensor_z="0.145"
  sensor_period_sec="$f{1/200.0}"
  sensor_name="imu1"
/>
```

Paste below the resulting `<vehicle>` block:

```
<vehicle name="r1" class="turtlebot3_burger">
    <init_pose>-2 -0.5 0</init_pose>  <!-- In global coords: x,y,
yaw(deg) -->

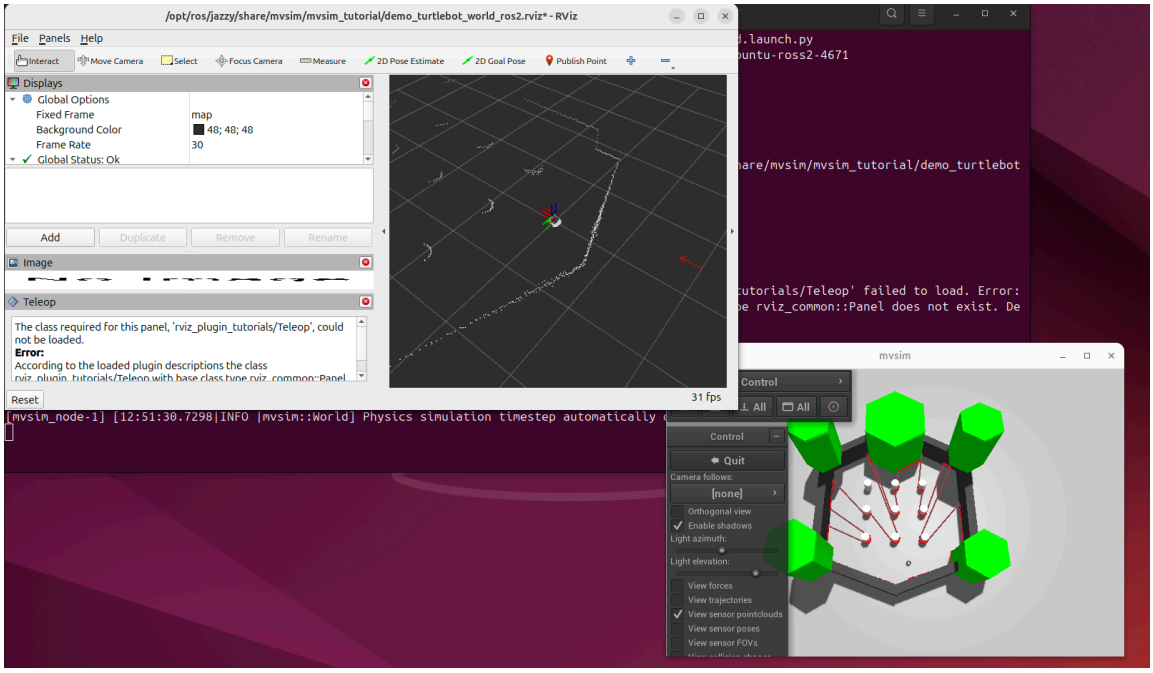
    <!-- 2D laser scanner (RPLIDAR A2), as in the TurtleBot3 class
-->
    <include file="../definitions/rplidar-a2.sensor.xml"
      raytrace_3d="true"
      sensor_x="-0.032" sensor_z="0.171" sensor_yaw="0"
      sensor_hz="10.0"
      sensor_name="laser1"
    />

    <!-- IMU, as in the TurtleBot3 class -->
    <include file="../definitions/imu.sensor.xml"
      sensor_x="0.0" sensor_y="0.05" sensor_z="0.145"
      sensor_period_sec="$f{1/200.0}"
      sensor_name="imu1"
    />
</vehicle>
```

4. Launch the simulation, and include a **screenshot** showing both the MVSim GUI and RViz2:

```
ros2 launch mvsim demo_turtlebot_world.launch.py
```

*La simulación arranca y carga el mundo modificado sin errores («World file load done»). En MVSim (abajo a la derecha) se ve el TurtleBot3 en la arena de columnas con los rayos del láser; en RViz2 (arriba a la izquierda), el robot con sus ejes de tf y los puntos del scan sobre las paredes y las columnas. El panel **Image** de RViz2 muestra «No Image», como era de esperar: hemos quitado la cámara, así que nadie publica en su topic. El error del panel **Teleop** tampoco tiene que ver con nuestros cambios: la configuración de RViz2 de la demo incluye un panel de un paquete de tutoriales (`rviz_plugin_tutorials`) que no está instalado, y en esta práctica movemos el robot publicando en `/cmd_vel`.*



4.3.3 The robot's sensors in ROS2

Each sensor simulated by MVSim publishes its readings in its own topic, with a message type suited to the kind of data it produces. Besides the data themselves, every sensor message starts with a **header** (`std_msgs/msg/Header`) that carries two pieces of information that are key in robotics:

- `stamp` : **when** the measurement was taken.
- `frame_id` : **where** it was taken, that is, the name of the reference frame in which the data are expressed (for a sensor, usually a frame attached to it). We will come back to this in section 4.3.6.

ASSIGNMENT 2: Sensor inventory

With the simulation running, find out in another terminal what each sensor of the robot publishes.

What to do?

1. Fill in the table below. *Hints:* `ros2 topic list -t` *gives you the topics together with their types;* `ros2 topic echo <topic> --once --no-arr` *prints a single message hiding its long arrays, so you can read its* `frame_id` *at a glance; and remember that there is a* `ros2 topic` *subcommand that measures the rate of a topic.* The first row is filled in for you, as an example.

Sensor	Topic	Message type	`frame_id`	Rate (Hz)
2D laser scanner	`/laser1`	`sensor_msgs/msg/LaserScan`	`laser1`	~10
IMU	`/imu1`	`sensor_msgs/msg/Imu`	`imu1`	~147 (configured: 200)
Wheel encoders (odometry)	`/odom`	`nav_msgs/msg/Odometry`	`odom` (`child_frame_id`: `base_link`)	~36 (configured: 50)

```
# Topics con su tipo de mensaje
ros2 topic list -t
# ...
# /imu1 [sensor_msgs/msg/Imu]
# /laser1 [sensor_msgs/msg/LaserScan]
# /odom [nav_msgs/msg/Odometry]
# ...

# Un solo mensaje sin los arrays largos, para leer el frame_id de su header
ros2 topic echo /laser1 --once --no-arr
ros2 topic echo /imu1 --once --no-arr     # header.frame_id: imu1
ros2 topic echo /odom --once --no-arr     # header.frame_id: odom, child_frame_id: base_link

# Frecuencia de publicación (Ctrl+C para pararlo)
ros2 topic hz /laser1                     # average rate: ~9.7–9.8
ros2 topic hz /imu1                       # average rate: ~147
ros2 topic hz /odom                       # average rate: ~35–36
```

2. Compare the rates you measured with the configuration of each sensor in the world file. Do they match? *Note: if your computer cannot keep the simulation in real time, MVSim prints a warning ("Simulation slower than real time") and the rates you measure will be somewhat lower.*

Láser: *en el mundo está configurado con* `sensor_hz="10.0"` *y medimos ~9,7–9,8 Hz, así que coincide.*
IMU: *está configurada con* `sensor_period_sec="${1/200.0}"` *(200 Hz) y medimos ~147 Hz, bastante menos. No es que la simulación entera vaya más lenta que el tiempo real: MVSim no ha dado el aviso «Simulation slower than real time», el nodo tiene* `realtime_factor: 1.0` *y el láser sí sale a su frecuencia. El límite está en el bucle del simulador, que corre a* `simul_rate: 100.0` *Hz (una vuelta cada 10 ms): para dar 200 Hz, la IMU tiene que generar* ***dos lecturas en cada vuelta****. Los sellos de tiempo de mensajes seguidos (* `ros2 topic echo /imu1 --field header.stamp.nanosec` *) lo confirman: llegan por parejas casi en el mismo instante (p. ej. 876.19 ms y 876.40 ms) y las parejas van separadas ~10 ms, pero no todas las*

vueltas producen dos lecturas (en una ventana de 60 ms salieron 10 mensajes, unos 165 por segundo). Por eso `ros2 topic hz` mide intervalos mínimos de 0,000 s y una media por debajo de 200 Hz: en la máquina virtual el simulador no da abasto con un sensor que publica más rápido que su propio bucle. Al láser (10 Hz, una lectura cada 10 vueltas) no le afecta.
***Odometría:** no se configura en el mundo, sino en el nodo de MVSim: `period_ms_publish_tf: 20.0`, es decir, 50 Hz para `/odom` y `/tf`. Medimos ~36 Hz porque esa publicación solo se comprueba en cada vuelta del bucle de simulación (cada ~10 ms): cuando han pasado 20 ms se publica en la siguiente vuelta, que con la irregularidad de la máquina virtual cae a veces a los 20 ms y a veces a los 30 ms (intervalos medidos entre 0,017 y 0,061 s).*
***Nota:** en `ros2 topic list` siguen apareciendo `/camera1/image_raw` y `/camera1_points` aunque hemos quitado la cámara. `ros2 topic info /camera1/image_raw` muestra `Publisher count: 0` y `Subscription count: 1`: el topic existe solo porque RViz2 está suscrito a él, pero nadie publica imágenes.*

4.3.4 The laser scanner

A 2D laser scanner publishes `sensor_msgs/msg/LaserScan` messages. Each message is a whole **scan**, that is, a set of K measurements $\mathbf{z} = [\mathbf{z}_1, \ldots, \mathbf{z}_K]$, each one $\mathbf{z}_k = [r_k, \alpha_k]^T$ in polar coordinates relative to the sensor, just like the observations of the `Laser2D` class of notebook 4.2.

However, the message does **not** store K pairs $[r_k, \alpha_k]^T$. The beams of a scanner are always emitted at the same, evenly spaced, angles: the first one at an angle `angle_min`, and each of the following ones `angle_increment` radians after the previous one. So, instead of the K angles, the message stores just these two values, and then **only the ranges**, in the `ranges` list, sorted by angle. The angle of each range is given by its position k in that list (counting from $k = 0$):

$$\alpha_k = \texttt{angle_min} + k \cdot \texttt{angle_increment}$$

ASSIGNMENT 3: Anatomy of a laser scan

What to do?

1. Find the command that shows the definition of the `sensor_msgs/msg/LaserScan` message type, that is, its fields together with the comments that explain them. *Hint: message types are **interfaces**, and there is a ROS2 subcommand for them.*

```
ros2 interface show sensor_msgs/msg/LaserScan
# std_msgs/Header header
# float32 angle_min          # start angle of the scan [rad]
```

```
# float32 angle_max            # end angle of the scan [rad]
# float32 angle_increment      # angular distance between
measurements [rad]
# float32 time_increment       # time between measurements
[seconds]
# float32 scan_time            # time between scans [seconds]
# float32 range_min            # minimum range value [m]
# float32 range_max            # maximum range value [m]
# float32[] ranges             # range data [m] (values < range_min
or > range_max should be discarded)
# float32[] intensities        # intensity data [device-specific
units]
```

2. Print a single message of `/laser1` without its long arrays, and fill in the table below, which relates its fields with the parameters of the `Laser2D` class of notebook 4.2. Give the angles in degrees.

```
ros2 topic echo /laser1 --once --no-arr
# header:
#   stamp:
#     sec: 1793621364
#     nanosec: 172189236
#   frame_id: laser1
# angle_min: -3.1415927410125732
# angle_max: 3.1415927410125732
# angle_increment: 0.00786381121724844
# time_increment: 0.0
# scan_time: 0.0
# range_min: 0.019999999552965164
# range_max: 16.0
# ranges: '<sequence type: float, length: 800>'
# intensities: '<sequence type: float, length: 0>'
```

`Laser2D` parameter (4.2)	**`LaserScan` field(s)**	**Value**
Field of view (`FOV`)	`angle_max - angle_min`	$\pi - (-\pi) = 2\pi$ rad = **360°** (from –180° to +180°)
Angular resolution (`resolution`)	`angle_increment`	0.0078638 rad = **0.4506°**
Max distance (`max_distance`)	`range_max`	**16.0 m** (and `range_min` = 0.02 m)
Number of beams	length of `ranges`	**800** $= \frac{2\pi}{0.0078638} + 1 = 799 + 1$

3. Using the expression of α_k given above, which index k of `ranges` corresponds, approximately, to the beam pointing straight ahead of the robot? And which directions do the first and the last readings point to?

El haz que apunta al frente del robot es el de ángulo $\alpha_k = 0$, es decir, el eje x del láser (que en el mundo tiene `sensor_yaw="0"`, así que mira hacia donde mira el robot). Despejando:
$\alpha_k =$ `angle_min` $+ k \cdot$ `angle_increment` $= 0 \Rightarrow k =$ − `angle_min` / `angle_increment` $= 3.14159 / 0.0078638 \approx$ ***399.5****.*
Ningún haz cae justo en 0: el frente queda entre ***k = 399*** *(α = −0.225°, un poco a la derecha) y* ***k = 400*** *(α = +0.225°, un poco a la izquierda). Cualquiera de los dos vale como «el de frente».*
La primera lectura (k = 0) apunta a $\alpha_0 =$ `angle_min` $= -180°$ y la última (k = 799) a $\alpha_{799} = -180° + 799 \cdot 0.4506° =$ `angle_max` $= +180°$: ***las dos apuntan hacia atrás****, en la misma dirección. Como el campo de visión es de 360° y el simulador incluye los dos extremos, el primer y el último haz se solapan (en los scans medidos los dos leen unos 0.6 m). Entre medias, los ángulos crecen en sentido antihorario (alrededor del eje Z): k = 200 apunta a la derecha del robot (≈ −90°) y k = 600 a su izquierda (≈ +90°).*

4.3.5 Sensor noise and configuration

As we modeled in notebooks 4.1 and 4.2, the measurements of a real sensor are not exact, and MVSim adds noise to the simulated ones too. For the laser scanner, `definitions/rplidar_a2.sensor.xml` defines a zero-mean Gaussian noise in the range of each beam, with standard deviation `range_std_noise` (set with the `sensor_std_noise` argument), and another one in its angle, with standard deviation `angle_std_noise_deg` .

ASSIGNMENT 4: Measuring the noise of the scanner

Keep the robot **standing still** (do not publish anything in `/cmd_vel`).

What to do?

1. Print the `ranges` of `/laser1` a few times and compare the value of the **same beam** in different scans. Write below the command you used, and the values you got. *Hint:* `ros2 topic echo /laser1 --once --field ranges` *prints only that field.*

```
# 10 scans seguidos con el robot parado, solo el campo ranges
for i in $(seq 10); do ros2 topic echo /laser1 --once --field
ranges; echo '---'; done > /tmp/scans.txt
# array('f', [0.5857509970664978, 0.6747241616249084,
0.6403195858001709, ...])   <- 800 valores por scan
```

Same beam in the 10 scans (values in m, extracted from `/tmp/scans.txt` with a short Python script):

Beam k	Angle	Scan 1 … 10	Std. dev.
0	−180°	0.586 0.592 0.590 0.676 0.679 0.598 0.699 0.651 0.633 0.629	0.042
100	−135°	0.652 0.649 0.667 0.654 0.642 0.662 0.658 0.675 0.662 0.643	0.011
200	−90°	1.623 1.608 1.622 1.602 1.611 1.604 1.607 1.618 1.608 1.611	0.008
300	−45°	3.020 2.990 2.960 2.971 2.995 3.013 2.991 3.026 3.004 3.051	0.027
400	0° (front)	4.061 4.047 4.050 4.039 4.050 4.046 4.060 4.074 4.027 4.049	0.013
500	+45°	4.338 4.343 4.359 4.363 4.341 4.353 4.351 4.365 4.360 4.347	0.010
600	+90°	2.720 2.703 2.717 2.705 2.723 2.710 2.713 2.720 2.724 2.745	0.012
700	+135°	1.005 1.064 0.966 1.044 1.016 1.064 1.006 1.016 0.993 1.029	0.031
799	+180°	0.638 0.637 0.628 0.630 0.607 0.616 0.624 0.625 0.638 0.617	0.010

Over the 800 beams (all of them return a reading in the 10 scans): mean std. dev. = **0.0154 m**, median = **0.0130 m**.

2. How much do those values vary? Is it consistent with the `range_std_noise` of the scanner?

*Con el robot quieto, el mismo haz no da nunca dos veces el mismo valor: varía unos **±1–2 cm** de un scan a otro (desviación típica mediana de 0.013 m sobre los 800 haces). Es coherente con el ruido configurado: el láser lo incluimos sin el argumento `sensor_std_noise`, así que en `rplidar-a2.sensor.xml` toma su valor por defecto, `<range_std_noise>${sensor_std_noise|0.01}</range_std_noise>`, es decir, σ_r = **0.01 m**. La variación tampoco depende de la distancia: a 1.6 m (k = 200) y a 4.3 m (k = 500) es prácticamente la misma (~0.01 m), como corresponde a un ruido gaussiano de media cero con σ fija, igual que el que añadíamos en el `Laser2D` del notebook 4.2.*
La media sale algo por encima (0.015 m) por unos pocos haces que varían bastante más (0.03–0.04 m, como k = 0, 300 o 700). Eso ya no lo explica el modelo gaussiano: k = 0 y k = 799 apuntan exactamente a la misma dirección y uno varía 4 cm y el otro 1 cm. Lo más probable es que sea un efecto de cómo simula MVSim el láser con `raytrace_3d="true"` (las distancias se calculan sobre imágenes de profundidad renderizadas, que tienen su propia discretización) y, en menor medida, del pequeño

ruido en el ángulo de cada haz (`angle_std_noise_deg` = 0.01°). Por eso la mediana, que no se deja arrastrar por esos pocos haces, es el valor que mejor estima σ_r.

ASSIGNMENT 5: Configuring the scanner

Now let's change the scanner. Edit the world file again, and add these two arguments to the `<include>` of `laser1`:

```
sensor_std_noise="0.10"
max_range="3.0"
```

What to do?

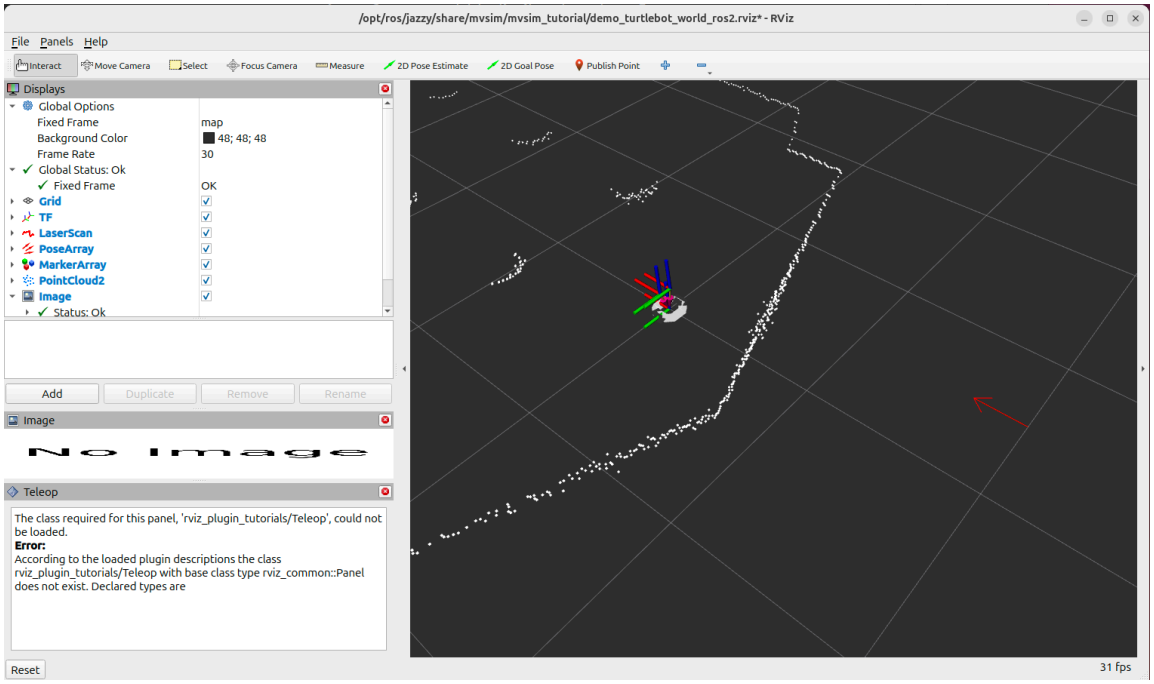
1. Paste below the resulting `<include>` block.

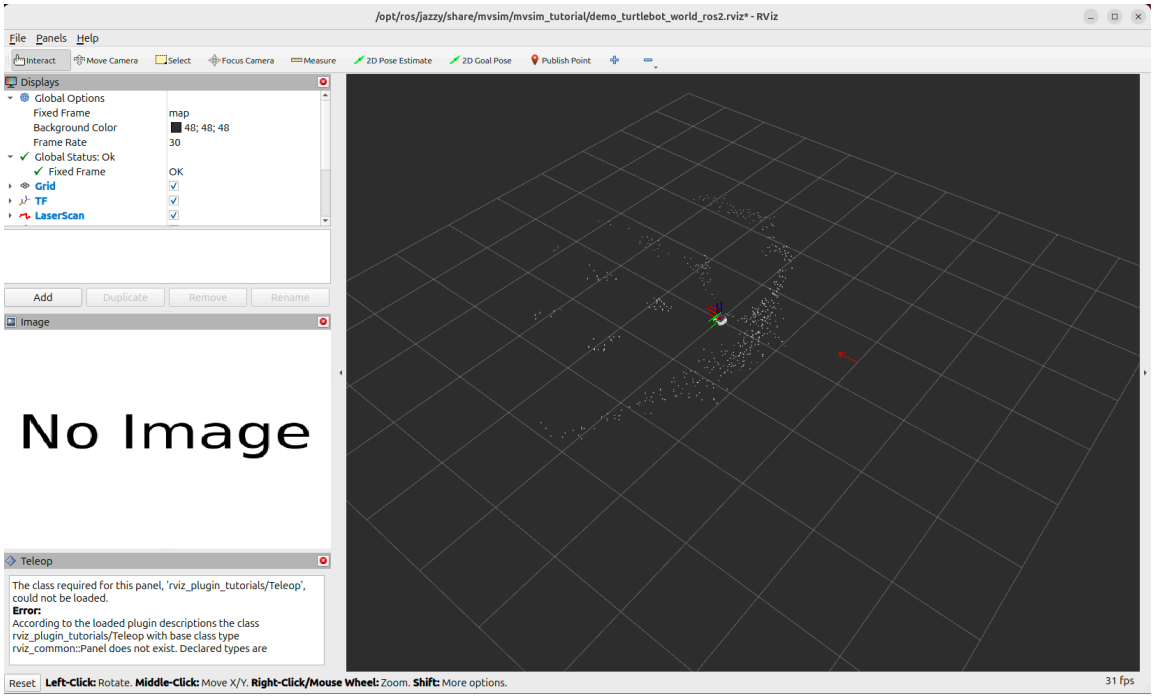
```
<!-- 2D laser scanner (RPLIDAR A2), as in the TurtleBot3 class -->
<include file="../definitions/rplidar_a2.sensor.xml"
  sensor_std_noise="0.10"
  max_range="3.0"
  raytrace_3d="true"
  sensor_x="-0.032" sensor_z="0.171" sensor_yaw="0"
  sensor_hz="10.0"
  sensor_name="laser1"
/>
```

2. Launch the simulation again and include a **screenshot** of RViz2 showing the scan with the new configuration. Compare it with the original one: what has changed, and why?

Arriba, el scan original (σ_r = 0.01 m, alcance 16 m); abajo, el nuevo (σ_r = 0.10 m, alcance 3 m), con la vista algo más alejada. Cambian dos cosas:

- ***Los puntos están mucho más dispersos.*** *Antes, las paredes y las columnas se veían como líneas finas y nítidas; ahora son nubes de puntos «gruesas», de unos ±10–20 cm de ancho, porque la desviación típica del ruido en distancia es diez veces mayor (`sensor_std_noise="0.10"` fija `range_std_noise` = 0.10 m). Las columnas pequeñas casi no se distinguen.*
- ***Desaparece todo lo que está a más de 3 m.*** *Antes se veían las paredes lejanas de la arena (a 4–5 m del robot); ahora solo se ven los obstáculos cercanos, porque `max_range="3.0"` limita el alcance. Los haces que no chocan con nada dentro de esos 3 m devuelven exactamente 3.0 (ver punto 3), y RViz2 no los dibuja: los trata como «sin retorno», como indica el comentario de `ranges` en la definición del mensaje.*





3. Print a single message of `/laser1`, first without its arrays and then only its `ranges`. Which value does `range_max` take now? And which value do the beams that do not hit anything within the maximum range take?

```
ros2 topic echo /laser1 --once --no-arr
# ...
# range_min: 0.019999999552965164
# range_max: 3.0
# ranges: '<sequence type: float, length: 800>'

ros2 topic echo /laser1 --once --field ranges
# array('f', [0.8736, 0.6880, 0.7547, ..., 3.0, 3.0, 3.0, ...])
```

```
#   k = 195..204 (~1.6 m, to the right):  1.656 1.659 1.669 1.673
1.517 1.520 1.597 1.749 1.753 1.549
#   k = 295..304 (~3 m, front-right):     2.505 2.812 2.801 2.958
2.652 2.954 2.927 3.000 2.836 3.000
#   k = 395..404 (front, wall at ~4 m):   3.000 3.000 3.000 3.000
3.000 3.000 3.000 3.000 3.000 3.000
#   k = 495..504 (front-left, ~4.3 m):    3.000 3.000 3.000 3.000
3.000 3.000 3.000 3.000 3.000 3.000
#   144 of the 800 beams are 3.000; no inf or nan values
```

`range_max` *vale ahora* ***3.0*** *m (antes 16.0), el valor que hemos puesto en* `max_range`*. Los haces que no chocan con nada dentro de ese alcance* ***no devuelven*** `inf` ***ni un valor vacío, sino exactamente*** `range_max` ***= 3.0****: en este scan son 144 de los 800, por ejemplo todos los del frente (k = 395–404), donde antes la pared estaba a ~4.05 m, y los de k = 495–504, donde estaba a ~4.35 m. Así que una lectura de 3.0 no significa «hay un obstáculo a 3 m», sino «no hay nada hasta 3 m».*
El ruido también se nota en los números: en k = 195–204, a ~1.6 m, los valores van de 1.52 a 1.75 m, cuando con el láser original apenas variaban unos centímetros (1.57–1.63 m).

4. Restore the original configuration of the scanner, removing those two arguments, before going on.

Thinking about it

Having explored the laser scanner, **answer the following questions**:

- The `Laser2D` class of notebook 4.2 returned each reading as a pair $[r_k, \alpha_k]^T$. What does a `LaserScan` message gain by storing only the ranges?

  La ***mitad de datos****. Los ángulos de los haces son siempre los mismos y están equiespaciados, así que no hace falta enviarlos: basta con* `angle_min` *y* `angle_increment` *(dos números por mensaje) y cada ángulo se recupera a partir de la posición k del rango,* α_k = `angle_min` + k·`angle_increment`*. Con nuestro láser son 800* `float32` *por scan (3.2 kB) en lugar de 1600 (6.4 kB), y a 10 scans por segundo eso es la mitad de ancho de banda en la red de ROS, de memoria y de espacio al grabar los datos. Además, refleja cómo funciona el sensor real, que dispara a ángulos fijos y solo mide la distancia.*

- Which problem would a mapping algorithm have if it treated the readings equal to `range_max` as real obstacles?

  Metería en el mapa ***obstáculos fantasma****. Como hemos visto en el Assignment 5, un haz que no choca con nada devuelve* `range_max`*; si el algoritmo lo tomara como un impacto real, dibujaría un arco de «pared» a 3 m del robot en todas las direcciones libres (en nuestro scan, 144 haces), y al moverse el robot iría llenando*

de paredes inexistentes las zonas abiertas, que además quedarían marcadas como no transitables para la planificación. Lo correcto es tratar esas lecturas como «sin retorno»: no aportan ningún obstáculo, como mucho que el espacio está libre hasta `range_max`*. Por eso la definición del mensaje dice que los valores fuera de [* `range_min` *,* `range_max` *] deben descartarse, y RViz2 no los dibuja.*

- What do we gain, and what do we lose, by increasing the number of beams of a scanner?

  ***Ganamos** resolución angular: los haces están más juntos, así que se detectan objetos más pequeños o más lejanos (que con pocos haces caerían entre dos de ellos) y los contornos del entorno salen con más puntos, lo que mejora los mapas y el ajuste de scans (como el ICP del notebook 4.1). **Perdemos** en coste: cada scan tiene más datos que enviar, guardar y procesar. Y en un láser rotatorio como el RPLIDAR A2, que dispara a un ritmo fijo (cada 125 µs, 8 kHz, según* `rplidar_a2.sensor.xml` *, que calcula* `nrays` *= (1/* `sensor_hz` *)/125·10^{-6}), más haces por vuelta solo se consiguen girando más despacio: con 10 Hz salen 800 haces, pero para tener 1600 habría que bajar a 5 scans por segundo, y el robot recibiría la información del entorno con más retraso entre scans.*

4.3.6 Where are the sensors? Reference frames and tf

The ranges of a `LaserScan` are expressed in the frame of the scanner (its `frame_id`, `laser1`), not in the frame of the robot, nor in the world one. To use them, for instance to place the scanned points on a map as we did in notebook 4.2, we need the pose of the scanner on the robot and the pose of the robot in the world, and compose them: a point $\mathbf{z}_c$ measured in cartesian coordinates in the scanner frame is expressed in the world frame as $\mathbf{x} \oplus \mathbf{x}_s \oplus \mathbf{z}_c$, being $\mathbf{x}$ the pose of the robot and $\mathbf{x}_s$ the pose of the sensor on the robot (see Fig. 3). In notebook 4.2 we took the shortcut of placing the scanner at the center of the robot, that is, $\mathbf{x}_s = [0, 0, 0]^T$.

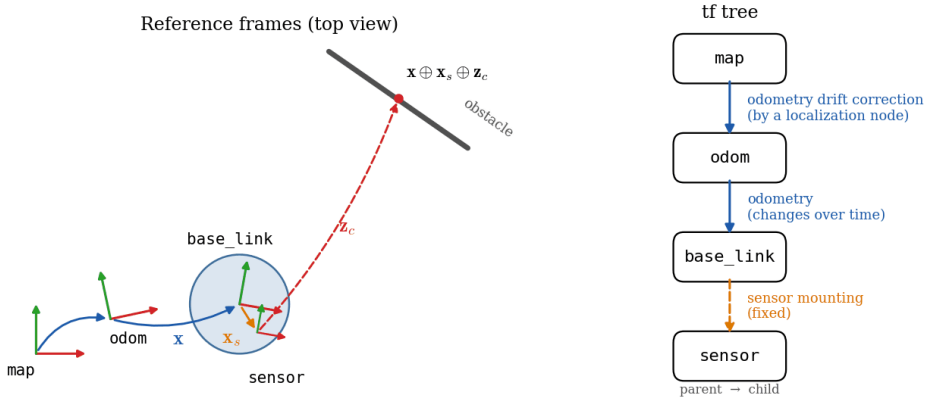


Fig. 3. Left, the reference frames of a mobile robot with a sensor, seen from above while the robot moves (with their x axes in red and their y axes in

green, as RViz2 draws them): `base_link` and the frame of the sensor move with the robot, while `map` and `odom` stay fixed in the world, and the point measured by the sensor is always expressed in the world frame as the same point. Right, the tf tree of these frames.

ROS2 manages all these poses with **tf2**. Its main ideas are:

- **Frames and transforms.** A *frame* is a reference system with a name, like `base_link` or `laser1`. A *transform* between a parent frame and a child frame is the pose of the child expressed in the parent: a translation and a rotation. It is the same idea as the poses $[x, y, \theta]^T$ of chapter 3, extended to 3D (ROS stores the rotations as quaternions, but its tools also show them as roll, pitch and yaw angles).
- **The tf tree.** Every frame has a single parent, so the frames form a tree, as in the right part of Fig. 3, and there is a single path between any two frames. To get the transform between two of them, tf2 composes the transforms along that path, which is exactly the composition of poses $\oplus$. For example, to get the pose of the sensor in the `odom` frame, tf2 follows the path `odom` → `base_link` → `sensor` of Fig. 3 and composes its two transforms: the pose of the robot in `odom`, $\mathbf{x}$, and the pose of the sensor on the robot, $\mathbf{x}_s$. The result is $\mathbf{x} \oplus \mathbf{x}_s$.
- **Who publishes the transforms.** The nodes publish the transforms they know in the `/tf` topic (those that change over time, e.g. the pose of the robot) and in `/tf_static` (the fixed ones, e.g. the pose of a sensor on the robot, although some nodes publish them in `/tf` too). Every transform carries a timestamp, so tf2 can give the transform at the instant a measurement was taken, given by the `stamp` of its header (section 4.3.3).
- **Standard frames.** Mobile robots in ROS follow the conventions of REP 105:
  - `map`: a frame fixed in the world. The pose of the robot in it does not drift, but it can jump when a localization algorithm corrects it.
  - `odom`: another frame fixed in the world, in which the odometry expresses the pose of the robot. This pose is continuous (no jumps), but it drifts over time. A localization node publishes the transform `map` → `odom` to correct that drift. In our simulation there is no such node, so MVSim publishes it as the identity.
  - `base_link`: a frame rigidly attached to the robot. The transform `odom` → `base_link` is the pose of the robot $\mathbf{x}$ given by the odometry.
  - `base_footprint`: the projection of `base_link` on the floor.
  - The frames of the sensors (e.g. `laser1`), which hang from the frames of the robot.

Fig. 4 shows the frames of our robot: on the left, the frames of its two sensors drawn by MVSim on the TurtleBot3 Burger (option *View sensor poses* of its GUI), and on the right, all its frames as RViz2 draws them with its `TF` display.

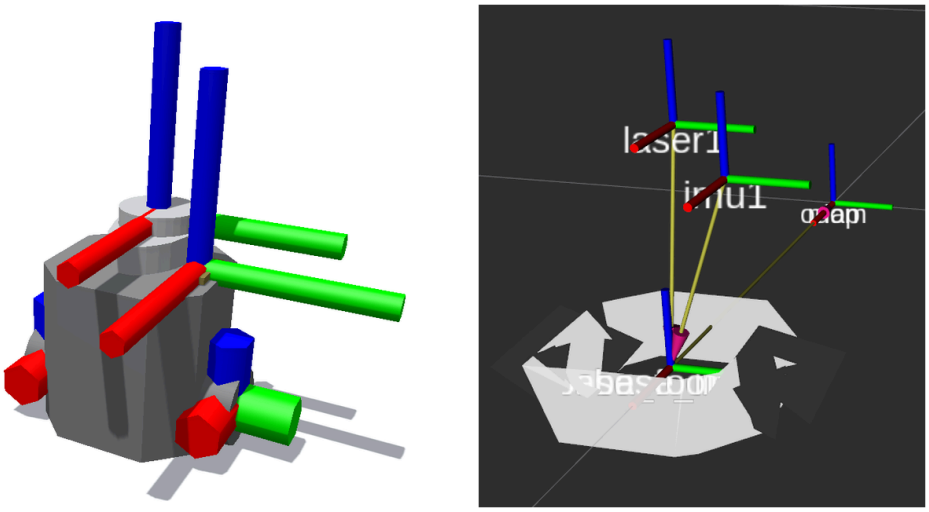


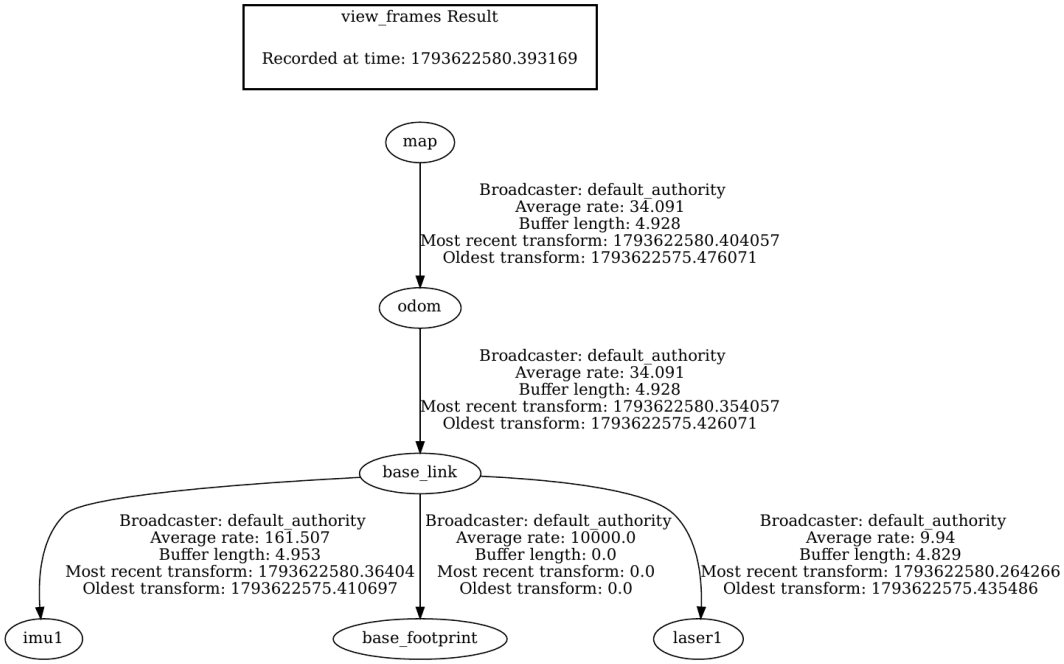
Fig. 4. Reference frames of the TurtleBot3 Burger: those of its sensors in MVSim (left), and the whole set in RViz2 (right). In the right image, some labels overlap: at the bottom, `base_link` and `base_footprint` (its projection on the floor, which in this robot coincides with it), and on the right, `map` and `odom`, which coincide in our simulation.

ASSIGNMENT 6: The tf tree of the robot

What to do?

1. With the simulation running, generate a diagram of the tf tree. *Hint: the `tf2_tools` package has an executable called `view_frames`, which listens to `/tf` for a few seconds and saves the tree in a PDF file in the current directory.* Write the command below and include a **screenshot** of the tree.

```
cd /tmp
ros2 run tf2_tools view_frames
# [view_frames]: Listening to tf data for 5.0 seconds...
# [view_frames]: Generating graph...
# [view_frames]:
Result:tf2_msgs.srv.FrameGraph_Response(frame_yaml="
#   imu1:           parent: 'base_link'  rate: 161.507
#   base_link:      parent: 'odom'       rate: 34.091
#   base_footprint: parent: 'base_link'  rate: 10000.000
most_recent_transform: 0.000000
#   odom:           parent: 'map'        rate: 34.091
#   laser1:         parent: 'base_link'  rate: 9.940 ")
# [view_frames]: Exporting graph in frames_2026-11-02_13.29.40.pdf
file...
```



Compare it with Fig. 3. Which frames does the tree of the TurtleBot3 have? Which of them are frames of sensors, and which is their parent? Which frame of the tree does not appear in the figure?

*El árbol tiene seis frames: **`map`** → **`odom`** → **`base_link`**, y de `base_link` cuelgan tres hijos: **`base_footprint`**, **`laser1`** e **`imu1`**.*
*• Los frames de sensores son **`laser1`** (el láser) e **`imu1`** (la IMU), y su padre es **`base_link`**: los sensores van montados en el robot, así que su pose se da respecto a él y se mueven con él. Es el mismo esquema que la Fig. 3 (`map` → `odom` → `base_link` → sensor), solo que con dos sensores.*
*• El frame que **no aparece en la figura es `base_footprint`**, la proyección de `base_link` sobre el suelo. Es una transformación fija: `view_frames` le da un `rate` de 10000 y un `most_recent_transform` de 0, que es como muestra las transformaciones estáticas (`/tf_static`), válidas para cualquier instante.*
• Las demás se publican en `/tf`, cada una a su ritmo: `map` → `odom` y `odom` → `base_link` a ~34 Hz (el `period_ms_publish_tf` de 20 ms del nodo, con el mismo retraso que medimos en `/odom` en el Assignment 2), y las de los sensores a la frecuencia de cada uno: `laser1` a ~10 Hz e `imu1` a ~160 Hz. Aunque la pose de un sensor sobre el robot no cambia, MVSim la vuelve a publicar en `/tf` con cada lectura, como avisa la introducción («some nodes publish them in `/tf` too»).

2. Print the transformation from `base_link` to `laser1`. *Hint: the `tf2_ros` package has an executable called `tf2_echo`, which takes the two frames as arguments.* Does it match the `<include>` of the scanner in the world file?

```
ros2 run tf2_ros tf2_echo base_link laser1
# At time 1793622583.857984304
# - Translation: [-0.032, 0.000, 0.171]
# - Rotation: in Quaternion (xyzw) [0.000, 0.000, 0.000, 1.000]
# - Rotation: in RPY (radian) [0.000, -0.000, 0.000]
# - Rotation: in RPY (degree) [0.000, -0.000, 0.000]
```

Sí, coincide exactamente con el `<include>` del láser en el mundo: `sensor_x="-0.032"` → traslación en x de −0.032 m (el láser está 3.2 cm por detrás del centro del robot), `sensor_y` no se da y vale 0 por defecto → y = 0, `sensor_z="0.171"` → z = 0.171 m (está arriba del todo, a 17 cm del suelo) y `sensor_yaw="0"` → rotación nula (cuaternión [0, 0, 0, 1], yaw = 0), es decir, el láser mira hacia el mismo lado que el robot. En el plano, la pose del láser sobre el robot es $\boldsymbol{x_s = [-0.032, 0, 0]^T}$.

3. Now print the transformation from `odom` to `base_link`, while you drive the robot from another terminal, publishing in `/cmd_vel`. What does it represent?

```
# Terminal 1: el robot avanza a 0.1 m/s girando a 0.3 rad/s
ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist "{linear: {x:
0.1}, angular: {z: 0.3}}"

# Terminal 2
ros2 run tf2_ros tf2_echo odom base_link
# - Translation: [0.027, 0.001, 0.000]   Rotation: in RPY (radian)
[0.000, -0.000, 0.074]
# - Translation: [0.120, 0.020, 0.000]   Rotation: in RPY (radian)
[0.000, -0.000, 0.338]
# - Translation: [0.202, 0.062, 0.000]   Rotation: in RPY (radian)
[0.000, -0.000, 0.594]
# - Translation: [0.271, 0.123, 0.000]   Rotation: in RPY (radian)
[0.000, -0.000, 0.851]
# - Translation: [0.323, 0.200, 0.000]   Rotation: in RPY (radian)
[0.000, -0.000, 1.111]
# - Translation: [0.354, 0.289, 0.000]   Rotation: in RPY (radian)
[0.000, -0.000, 1.371]
```

Representa la ***pose del robot estimada por la odometría****, $x = [x, y, \theta]^T$, expresada en el frame `odom`: dónde cree el robot que está respecto al punto en el que empezó, a partir de lo que han girado sus ruedas. Por eso empieza en (0, 0, 0) aunque el mundo coloca al robot en (−2, −0.5) m: el origen de `odom` es la pose inicial del robot. Mientras se mueve, la traslación y el yaw crecen de forma continua, sin saltos: el robot avanza y gira a la izquierda (θ pasa de 0.07 a 1.37 rad, unos 0.26 rad por segundo). Los puntos describen un arco de circunferencia: en todos ellos $x \approx R \sin\theta$ e $y \approx R(1 - \cos\theta)$ con $R \approx 0.36$ m, cerca del radio que corresponde a las velocidades enviadas, $R = v/\omega = 0.1/0.3 = 0.33$ m. Es el movimiento a velocidad constante del modelo de velocidad del*

capítulo 3. Al ser odometría, esta pose acumulará deriva con el tiempo; en una simulación sin deslizamientos casi no se nota, pero en un robot real es la razón de que exista la transformación `map` → `odom` para corregirla.

4. Finally, let's check that tf2 composes poses just as we do. Stop the robot and print the transformation from `odom` to `base_link`: its translation x, y and its yaw angle θ (the last value of `Rotation: in RPY (radian)`) give you the pose of the robot on the plane, $\mathbf{x} = [x, y, \theta]^T$. From step 2 you already know the pose of the scanner on the robot, $\mathbf{x}_s = [x_s, y_s, \theta_s]^T$. Compute by hand the pose of the scanner in `odom`, that is:

$$\mathbf{x} \oplus \mathbf{x}_s = \begin{bmatrix} x + x_s \cos\theta - y_s \sin\theta \\ y + x_s \sin\theta + y_s \cos\theta \\ \theta + \theta_s \end{bmatrix}$$

   whose height is just the sum of the heights z of both transformations. Then print the transformation from `odom` to `laser1`: does it match your result?

```
# Robot parado (MVSim lo para solo 0.2 s después de dejar de
recibir /cmd_vel)
ros2 run tf2_ros tf2_echo odom base_link
# - Translation: [0.350, 0.448, 0.000]
# - Rotation: in RPY (radian) [0.000, -0.000, 1.816]

ros2 run tf2_ros tf2_echo base_link laser1
# - Translation: [-0.032, 0.000, 0.171]
# - Rotation: in RPY (radian) [0.000, -0.000, 0.000]

ros2 run tf2_ros tf2_echo odom laser1
# - Translation: [0.358, 0.417, 0.171]
# - Rotation: in RPY (radian) [0.000, -0.000, 1.816]
```

*Datos: pose del robot en `odom`, **x** = [0.350, 0.448, 1.816]^T (z = 0), y pose del láser sobre el robot, $\mathbf{x}_s$ = [−0.032, 0, 0]^T (z = 0.171). Con cos(1.816) = −0.2428 y sin(1.816) = 0.9701:*

- *x: 0.350 + (−0.032)·(−0.2428) − 0·0.9701 = 0.350 + 0.0078 = **0.358 m***
- *y: 0.448 + (−0.032)·0.9701 + 0·(−0.2428) = 0.448 − 0.0310 = **0.417 m***
- *θ: 1.816 + 0 = **1.816 rad** (104.08°)*
- *z: 0 + 0.171 = **0.171 m***

***x ⊕ $\mathbf{x}_s$ = [0.358, 0.417, 1.816]^T**, que coincide con lo que da `tf2_echo odom laser1`: traslación [0.358, 0.417, 0.171] y yaw 1.816 rad. tf2 ha recorrido el camino `odom` → `base_link` → `laser1` del árbol y ha compuesto las dos transformaciones, igual que hacíamos a mano con ⊕.*

El resultado tiene sentido: el robot mira a 104°, casi hacia +y, y el láser está 3.2 cm por detrás de su centro. «Detrás» apunta a 104° + 180° = 284°, casi hacia −y, así que el

láser queda 3.1 cm más abajo en y (0.448 → 0.417) y apenas se desplaza en x (+0.8 cm). La orientación no cambia porque el láser no está girado respecto al robot (θ_s = 0).

OPTIONAL: Observing a landmark with tf

tf2 is not only for the frames of the robot: any node can publish a frame, for example one for a landmark of the map. Then, tf2 can tell us where that landmark is with respect to the scanner, that is, the observation that the scanner should provide, which is the predicted observation $\hat{\mathbf{z}} = \mathbf{h}(\mathbf{x}, \mathbf{m})$ of notebook 4.1.

As landmark, let's take the center of the white column at $(-1.1, 0)$ m in the world. Note that tf2 does not know the world frame of MVSim: the odometry starts at zero where the robot starts, which the world file places at $(-2, -0.5)$ m with a yaw of 0°. So, in the `odom` frame (and in `map`, which coincides with it here), that column is at $(0.9, 0.5)$ m.

1. Launch the simulation again, so the robot is at its initial pose, and, without moving it, publish a static frame called `landmark` at that position from a new terminal:

```
ros2 run tf2_ros static_transform_publisher --x 0.9 --y 0.5 --
frame-id odom --child-frame-id landmark
```

Then print the transformation from `laser1` to `landmark`, and, from its translation, compute the range and bearing $[r, \alpha]^T$ of the landmark as seen from the scanner.

```
# TODO: your command here
```

Your answer here!

2. Which index k of `ranges` points to the landmark? Print the `ranges` of `/laser1` and compare the reading of that beam with r. Why are they not equal?

```
# TODO: your command here
```

Your answer here!

OPTIONAL: Recording and replaying sensor data

Real robots record their sensor data so they can be processed later, offline, as many times as needed: that is how most robotics datasets are built. ROS2 does it with **rosbags**.

1. While you drive the robot around, record the topics of the laser scanner, the IMU and the odometry, together with `/tf` and `/tf_static`, using `ros2 bag record`. Then stop the recording and show its content with `ros2 bag info`.
2. Close the simulation, play the bag back with `ros2 bag play`, and check that those topics are being published again, even though the simulator is no longer

running.

Write below the commands you used and the output of `ros2 bag info`.

Your answer here!

AI Appendix

A.1 How I used AI

- **Model/tool:**

  Un asistente basado en IA generativa (Claude Code con Claude Opus 5.5).

- **What I asked AI to help with (bullets):**
  - *Guiarme paso a paso con los comandos de ROS2 de cada assignment, que yo ejecutaba en la máquina virtual y cuya salida le pasaba.*
  - *Aplicar con `sed` los cambios del archivo del mundo (Assignments 1 y 5) y restaurarlo después.*
  - *Interpretar las salidas: frecuencias medidas frente a las configuradas, campos del `LaserScan`, ruido del láser y transformaciones de tf.*
  - *Un pequeño script de Python para comparar el mismo haz en 10 scans y para resumir los `ranges` con alcance de 3 m.*
  - *Redactar las respuestas y comprobar a mano la composición $x \oplus x_s$.*
- **Best prompt I used (paste):**
- **What I kept vs. changed from the AI output:**

  Todos los valores del notebook salen de comandos ejecutados en mi simulación, no de la IA. Su primera explicación de por qué la IMU daba ~147 Hz en lugar de 200 (que `ros2 topic hz` no daba abasto) resultó incompleta: al mirar los sellos de tiempo de los mensajes se vio que llegan por parejas en cada vuelta de 10 ms del simulador y que no siempre hay dos, así que la cambié. También tuve que repetir el Assignment 5, porque el primer intento de editar el mundo no se aplicó (`range_max` seguía en 16.0) y lo detecté en la salida. La composición de poses la comprobé contra `tf2_echo odom laser1`.

A.2 AI review

Ask one generative AI to review your memo using this prompt:

```
Act as a technical reviewer of this mobile robotics lab
notebook. Read my memo and return exactly 5 bullet points:

- two on theory (key ideas),
- two on practice (relevant ROS2 concepts and tools used, not
particular commands or topics),
```

```
- one on how to improve the work (most critical aspect:
limitations, parameter choices, robustness, possible
mistakes).

Keep each bullet ≤15 words and make them specific to my work.
```

Paste its **5** items here:

AI used for the review: the same AI assistant (Claude Code with Claude Opus 5.5).

- **(Theory 1)** AI claim:

  Un scan guarda solo rangos; los ángulos salen de angle_min y angle_increment.
- **(Theory 2)** AI claim:

  tf2 compone odom→base_link→laser1 igual que $x \oplus x_s$: comprobado numéricamente.
- **(Practice 1)** AI claim:

  Inspección con topic list/echo/hz/info: tipos, frame_id, frecuencias y publicadores reales.
- **(Practice 2)** AI claim:

  Configuración por argumentos del mundo XML; relanzar la simulación para aplicarla.
- **(Improvement)** AI claim:

  Ruido estimado con solo 10 scans; usar más y descartar lecturas iguales a range_max.

Finally, **do you agree with them?**, **would you have selected others?**

Estoy de acuerdo con los cinco. Los dos teóricos son las ideas centrales de la práctica: el formato compacto del `LaserScan` *(y el detalle de que el primer y el último haz se solapan con 360°) y que tf2 no es más que composición de poses. La mejora también es real: con 10 muestras la desviación típica de cada haz tiene un error relativo de ~24 % (1/√(2·9)), y por eso usé la mediana sobre los 800 haces; con más scans se podría separar mejor el ruido gaussiano de los pocos haces anómalos. Habría añadido un punto de práctica sobre las frecuencias: un sensor que publica más rápido que el bucle del simulador (IMU a 200 Hz con* `simul_rate` *de 100 Hz) no llega a su frecuencia en una máquina virtual, y conviene medirlo con los sellos de tiempo y no solo con* `ros2 topic hz`*.*